Computer Networks: Course Project Report

# Deep Reinforcement Learning-Based Intelligent Packet Scheduler for QoS-Aware Network Traffic Management

*A Deep Q-Network that assigns traffic classes to 5G sub-6 GHz, 5G mmWave and Wi-Fi 6E links*

---

## Team Members


| S. No. | Name | Register Number |
|---|---|---|
| 1 | Shreyas Ajit Parulekar | 25BCE2877 |
| 2 | Mohd Arham | 25BCE2530 |
| 3 | Aiden Trindade | 25BCE2876 |


Course: Computer Networks Slot: B1+TB1


Guide: Prof. Rajasekar VR

Department: BTECH CSE

School: SCOPE

Institute: Vit Vellore


Academic Year: 2026

## 1. Problem Statement

A modern user device is often connected to more than one radio network at once: a 5G sub-6 GHz macro cell, a 5G millimetre-wave (mmWave) small cell and a Wi-Fi 6E access point. At the same time it carries traffic with very different needs. Voice and other ultra-reliable low-latency (URLLC) traffic must leave within a few milliseconds, video (eMBB) needs steady throughput, and bulk data can wait. The scheduler at this point has to decide, every time slot, which traffic queue to serve and over which link.

Classical schedulers answer only half of that question. Strict Priority, Round Robin and Weighted Fair Queuing (WFQ) [4] decide which queue goes next using fixed rules and, in their classic form, ignore the state of the links underneath. The links do not stay still. A mmWave link offers the highest data rate but loses most of its capacity when a hand, body or building blocks the line of sight [5], and unlicensed Wi-Fi capacity changes with contention. A fixed rule tuned for one set of conditions performs badly when those conditions change. Strict Priority, for example, can starve best-effort traffic completely when high-priority load is steady.

The goal of this project is a scheduler that learns, from experience, a joint queue-and-link policy that keeps each traffic class within its latency target (SLA), avoids buffer overflow, and keeps working when the mmWave link is blocked. We frame the problem as a Markov decision process and solve it with a Deep Q-Network (DQN) [1].

## 2. Background

In reinforcement learning (RL), an agent takes an action in each state, receives a reward, and adjusts its policy to maximise long-run reward. Tabular RL breaks down when the state space is large or continuous, as it usually is in networking. Mnih et al. [1] showed that a neural network can approximate the action-value function Q(s, a) if training is stabilised with an experience replay buffer and a separate, slowly updated target network. Mao et al. [2] applied deep RL to resource scheduling (DeepRM) and reported performance comparable to state-of-the-art heuristics, and Luong et al. [3] survey deep RL in communications and networking, including network access, rate control and traffic routing. Our project applies the same DQN approach to multi-class packet scheduling over heterogeneous links.

## 3. System Architecture

The implementation has three parts: a Gymnasium simulation environment (envs/hetnet_env.py), the DQN agent and three baseline schedulers (models/), and a Flask server with a browser dashboard (app.py, frontend/). Figure 1 shows the decision loop.

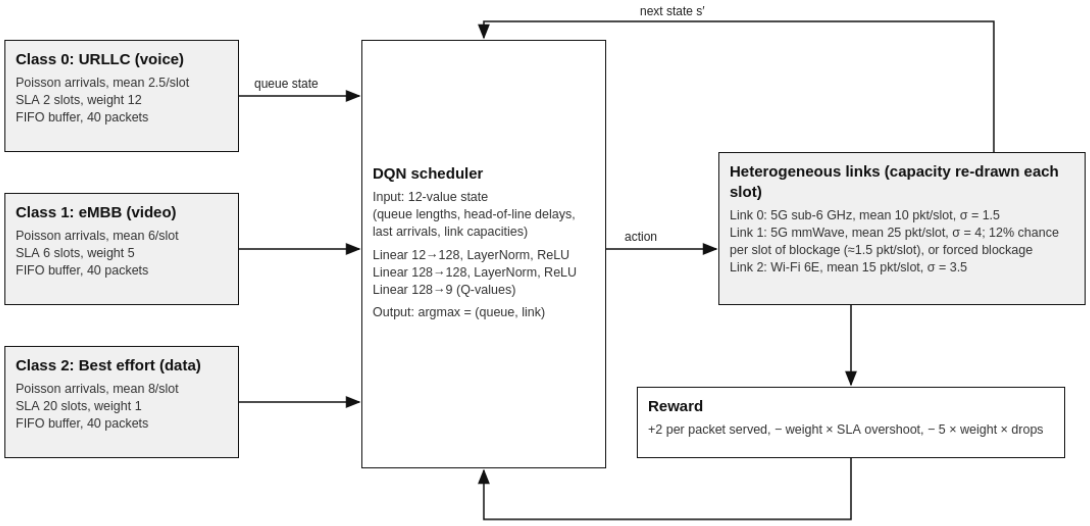



Figure 1. Decision loop of the scheduler. Every slot, the DQN reads the queue and link state and picks one (queue, link) pair.

### 3.1 Network environment

Time is divided into slots. Each slot, packets of three classes arrive according to a Poisson process and join a first-in, first-out buffer of 40 packets per class. Packets that arrive at a full buffer are dropped. Table 1 lists the class parameters. The SLA target is the maximum head-of-line (HOL) delay, in slots, before the class counts as late; the weight scales the penalty for lateness and drops.

| Traffic class | Mean arrivals / slot | SLA target (slots) | Penalty weight |
|---|---|---|---|
| 0: URLLC (voice) | 2.5 | 2 | 12 |
| 1: eMBB (video) | 6.0 | 6 | 5 |
| 2: Best effort (data) | 8.0 | 20 | 1 |

Table 1. Traffic classes as configured in hetnet_env.py.

Three links are available, and the capacity of each (packets per slot) is redrawn at random every slot. 5G sub-6 GHz has a mean of 10 with Gaussian noise ($\sigma = 1.5$) and is the steady fallback. 5G mmWave has a mean of 25 ($\sigma = 4$), but in any slot there is a 12% chance of blockage, which drops it to about 1.5 packets per slot. The user can also force a continuous blockage. Wi-Fi 6E has a mean of 15 with heavier noise ($\sigma = 3.5$) to represent contention. Only one queue is served per slot, over one link.

### 3.2 State, action and reward

- **State (12 values):** queue length of each class, HOL delay of each class, arrivals in the last slot for each class, and the three link capacities seen in the last slot.
- **Action (9 choices):** one (queue, link) pair, encoded as action = queue × 3 + link.
- **Reward:** +2 for every packet served; minus weight × (HOL delay − SLA target) for each class that is over its target; minus 5 × weight for every dropped packet. A late or dropped URLLC packet therefore costs 12 times as much as a best-effort one.

The link capacities in the state are the values from the previous slot. Because capacity is redrawn every slot, the scheduler cannot see the exact capacity it will get, only the recent history. This matches a real scheduler, which acts on channel reports that are already slightly out of date.

### 3.3 DQN model

The Q-network is a fully connected network: 12 inputs, two hidden layers of 128 units, each followed by Layer Normalisation and ReLU, and 9 outputs, one Q-value per action. At run time the scheduler picks the action with the highest Q-value. Following [1], training uses an experience replay buffer of 20,000 transitions and a target network that is copied from the policy network every 10 episodes.

## 4. Methodology

### 4.1 Training

Training (train.py) runs episodes of 200 slots; the script defaults to 300 episodes. The agent explores with an ε-greedy policy, with ε starting at 1.0 and multiplied by 0.993 after each episode down to a floor of 0.05. After every slot, a batch of 64 transitions is sampled from the replay buffer and the network is updated with the Adam optimiser (learning rate 0.001), discount factor $\gamma = 0.99$, mean-squared TD error and gradient clipping at norm 1.0. An episode ending at the time limit is treated as truncated, not terminal, so the agent still bootstraps from the final state.

The random 12% per-slot blockage rarely lasts more than one or two slots, so an agent trained only on it never learns what to do when mmWave stays down. To fix this, 40% of training episodes include a forced blockage. Half of these are blocked for the whole episode; the other half are blocked for a window of 30 to 119 slots starting at a random point, so the agent also sees the link fail and recover.

### 4.2 Baselines and evaluation

The DQN is compared against three classical schedulers (models/baselines.py). Strict Priority always serves the highest-priority non-empty queue. Round Robin cycles through the three queues in turn. Weighted Fair Queuing serves the queue with the largest backlog after weighting by 0.5, 0.3 and 0.2. All three send on the link with the highest capacity in the last observed slot.

Every scheduler is run on identical traffic: the environment is reset with the same random seed for each policy, so arrivals and channel conditions match slot for slot. We ran evaluate.py with 10 seeds of 300 slots each (3,000 slots per scheduler), in two scenarios: normal conditions, and mmWave blocked for the whole episode. We report total reward, packets served, packets dropped and SLA violations. An SLA violation is counted once for each slot and each class whose HOL delay is over its target.

## 5. Results

| Scheduler | Total reward | Served | Dropped | SLA violations |
|---|---|---|---|---|
| **Normal conditions** | | | | |
| DQN (trained) | **83,701** | **48,243** | **1,127** | 346 |
| Weighted Fair Queuing | 77,556 | 48,091 | 1,297 | 544 |
| Round Robin | 48,791 | 44,900 | 4,442 | **289** |
| Strict Priority | −153,526 | 27,630 | 21,605 | 2,350 |
| **mmWave blocked** | | | | |
| DQN (trained) | **21,028** | **41,396** | **7,763** | **920** |
| Weighted Fair Queuing | −6,470 | 41,334 | 7,784 | 1,607 |
| Round Robin | −85,745 | 35,273 | 13,845 | 1,028 |
| Strict Priority | −315,069 | 26,460 | 22,773 | 2,754 |

Table 2. Totals over 10 seeds × 300 slots per scheduler (python evaluate.py --seeds 10). Best value in each column in bold.

**Normal conditions.** The DQN has the highest total reward, 7.9% above WFQ. It drops 13% fewer packets than WFQ and has 36% fewer SLA violations. Round Robin has the fewest SLA violations (289 against the DQN's 346), but it drops four times as many packets (4,442), because it gives the heavily loaded best-effort queue the same share of slots as the lightly loaded URLLC queue. Strict Priority shows the starvation problem from Section 1: it serves the best-effort queue in only 3.1% of slots, its mean best-effort HOL delay is 51.5 slots against a target of 20, and it drops 21,605 packets.

**mmWave blocked.** With mmWave down, the best remaining link (Wi-Fi 6E, mean 15) has less capacity than the mean offered load of 16.5 packets per slot, so every scheduler must drop packets. The DQN and WFQ serve and drop almost the same number. The difference is in latency: the DQN has 43% fewer SLA violations than WFQ, and it is the only scheduler with a positive total reward. Logging the chosen actions shows how. The baselines follow the last observed capacity, and because Wi-Fi capacity is noisy they switch to the slower sub-6 GHz link in 10% of slots. The DQN stays on Wi-Fi in 98.6% of slots, which is the better choice on average because capacity is redrawn every slot and last slot's value says little about the next one.

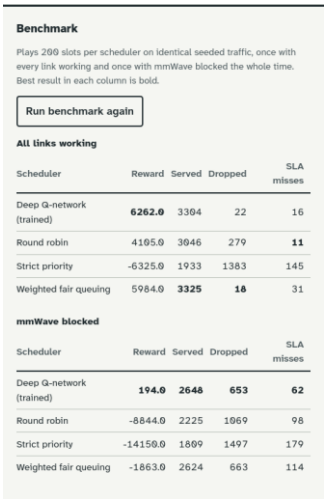
**Effect of blockage training.** The project also contains an earlier checkpoint, drl_hetnet_model_v1_no_blockage.pth (from its name, trained without the forced-blockage episodes). On the same 10-seed test it scores slightly higher in normal conditions (reward 86,575), but with mmWave blocked its reward falls to −5,031,430 and it serves only 3,751 packets, because it still picks the blocked mmWave link in 97.7% of slots. The forced-blockage episodes described in Section 4.1 are what make the final model work in the blocked scenario.

## 6. Proof of Concept

The Flask server (python backend/app.py) serves a dashboard at localhost:5000 that runs the simulator slot by slot. It shows the three buffers, link capacities, the (queue, link) choice for each slot and a live HOL-delay chart with each class's SLA as a dashed line. Schedulers, arrival rates and mmWave blockage can be changed while it runs. Figure 2 was captured from the running application after 66 slots with the trained DQN, with mmWave blocked from about slot 40.




Figure 2. Live dashboard. mmWave is blocked (2.0 packets per slot) and the DQN is draining the eMBB queue over Wi-Fi 6E. HOL delays stay near or under each class's SLA line.



**Benchmark**

Plays 200 slots per scheduler on identical seeded traffic, once with every link working and once with mmWave blocked the whole time. Best result in each column is bold.

Run benchmark again

**All links working**

| Scheduler | Reward | Served | Dropped | SLA misses |
|---|---|---|---|---|
| Deep Q-network (trained) | **6262.0** | 3304 | 22 | 16 |
| Round robin | 4165.0 | 3046 | 279 | **11** |
| Strict priority | -6325.0 | 1933 | 1383 | 145 |
| Weighted fair queuing | 5984.0 | **3325** | **18** | 31 |

**mmWave blocked**

| Scheduler | Reward | Served | Dropped | SLA misses |
|---|---|---|---|---|
| Deep Q-network (trained) | **194.0** | **2648** | **653** | **62** |
| Round robin | -8844.0 | 2225 | 1069 | 98 |
| Strict priority | -14150.0 | 1809 | 1497 | 179 |
| Weighted fair queuing | -1863.0 | 2624 | 663 | 114 |

Figure 3. The dashboard's built-in benchmark: one 200-slot run per scheduler on the same seeded traffic, once with all links working and once with mmWave blocked.

The numbers match a separate run of benchmark() in evaluate.py with the same seed. With mmWave blocked the DQN is best in every column. With all links working it leads on reward only: WFQ serves slightly more and drops fewer, and Round Robin has fewer SLA misses. The 10-seed totals in Table 2 are the more reliable comparison.

## 7. Limitations and Future Work

- All results come from simulation, and the link models are simple random processes. Training on measured 5G and Wi-Fi traces and testing in ns-3 are the natural next steps.
- Only one queue is served per slot, over one link. The comparison also uses a single trained model; repeating training with several seeds would show how much results vary between runs.

## References


[1] V. Mnih, K. Kavukcuoglu, D. Silver et al., "Human-level control through deep reinforcement learning," Nature, vol. 518, no. 7540, pp. 529–533, 2015. doi:10.1038/nature14236

[2] H. Mao, M. Alizadeh, I. Menache and S. Kandula, "Resource Management with Deep Reinforcement Learning," in Proc. 15th ACM Workshop on Hot Topics in Networks (HotNets), 2016, pp. 50–56. doi:10.1145/3005745.3005750

[3] N. C. Luong, D. T. Hoang, S. Gong, D. Niyato, P. Wang, Y.-C. Liang and D. I. Kim, "Applications of Deep Reinforcement Learning in Communications and Networking: A Survey," IEEE Communications Surveys & Tutorials, vol. 21, no. 4, pp. 3133–3174, 2019. doi:10.1109/COMST.2019.2916583

[4] A. Demers, S. Keshav and S. Shenker, "Analysis and Simulation of a Fair Queueing Algorithm," ACM SIGCOMM Computer Communication Review (Proc. SIGCOMM '89), vol. 19, no. 4, pp. 1–12, 1989. doi:10.1145/75247.75248

[5] S. Rangan, T. S. Rappaport and E. Erkip, "Millimeter-Wave Cellular Wireless Networks: Potentials and Challenges," Proceedings of the IEEE, vol. 102, no. 3, pp. 366–385, 2014. doi:10.1109/JPROC.2014.2299397